Milestone 4: A No-Limit Range-vs-Range Postflop Solver

holdem-cfr walkthrough

The pivot, and what it changes

The project now targets heads-up no-limit hold'em. That game has roughly 10^{161} decision points, against roughly 10^{14} for limit. More importantly, it has a continuous action space: a player can bet any amount up to their stack. Every serious no-limit system therefore makes two simplifications.

The first is a **bet-size abstraction**. The tree only contains a menu of sizes, such as a third of the pot, three quarters of the pot, and all-in. The solver's answer is an equilibrium *of that tree*. It is not an equilibrium of the real game, where an opponent can pick any size off the menu.

The second is **subgame solving**. Instead of solving the whole game, you solve the part that is in front of you, from the current street to the end, given ranges that describe what each player could hold. This milestone builds that core.

The tool takes a board, two ranges, a pot, stacks and a bet menu. It returns an equilibrium strategy for every hand at every decision, and an **exact** measurement of how exploitable that strategy is within the tree. Commercial tools such as PioSolver and GTO Wizard are built on this foundation.

Ranges, not hands

At the river, neither player knows the other's cards. What each player does know is the other's **range**: a probability weight on each of the 1,326 possible two-card combos. The weights come from how the hand was played so far. A player who raised before the flop and bet the flop has a range concentrated on strong hands and draws.

The input format follows poker convention:

- `QQ+` means QQ, KK and AA. - `ATs+` means AT through AK, suited. - `KQo-K9o` means the offsuit K9 through KQ. - `AsKs` is one specific combo. - `AA:0.5` gives AA half weight.

The parser is in `range.rs`. Its tests check textbook combo counts: 6 for a pair, 4 for a suited hand, 12 for an offsuit hand, 18 for `QQ+`.

Combos are indexed 0..1325 in *colex* order. For card indices hi > lo, the index is hi(hi − 1)/2 + lo. That is the same combinatorial-number-system idea the hand evaluator used for rank counts, applied here to pairs of cards.

The public tree

Split the game state into **public** information (the board, every bet and both players' investments) and **private** information (each player's hand). The public part forms a tree that does not depend on the cards, so `tree.rs` builds it once.

Player 0 is out of position (OOP) and acts first on every street. Player 1 is in position (IP).

Bet sizes are fractions of the pot. Raise sizes are fractions of the pot *after calling*, so a pot-sized raise facing a pot-sized bet into 100 is "call 100, then raise by 300, to 400".

The builder also enforces the rules that make a betting tree legal:

- A raise must be at least as large as the previous bet or raise increment. - Any size that would exceed the stack becomes a single all-in. - Duplicate sizes collapse into one action. - A per-street cap limits the total number of bets and raises. - After an all-in and a call, the remaining streets are dealt with no betting.

Every action stores the player's *total* chips put in after the action. That means every node knows the pot without replaying its history.

The tree also has **chance nodes**: in a turn spot, each of the 48 possible river cards leads to its own river subtree. The unit tests check the actual numbers: pot-sized raises, the minimum-raise rule, collapsing to all-in, the cap, and the all-in runout.

Vectorized CFR: one traversal for every hand at once

Milestones 1 and 2 ran CFR one history at a time. Here an information set is a pair (public node, private hand). The obvious approach would be to deal every pair of hands and walk the tree separately for each. With hundreds of hands per range, that means repeating the same public walk tens of thousands of times.

Vectorized CFR instead walks the public tree **once** per player per iteration, carrying vectors indexed by hand. Two vectors do the work:

- **Going down**, the solver passes the *opponent's reach vector*. Entry o is the probability that the opponent holds hand o and has played to this node. At an opponent decision node, each child receives the reach multiplied elementwise by the probability of that action. - **Coming back up**, each node returns the *traverser's counterfactual value vector*. Entry h is the value of holding hand h here, weighted by the opponent reach described above.

At the traverser's own nodes, regret matching runs independently for every hand, because each hand is its own information set. This is the exact algorithm from milestone 1, with the per-hand bookkeeping turned into loops over arrays.

Storage is *action-major*. The regret for action a and hand h at a node with offset o lives at index `o + a·n + h`. Every inner loop therefore walks contiguous memory, which the CPU prefetches and the compiler can vectorize.

Chance nodes need care with **card removal**, because a river card cannot be one that either player holds. For a fixed pair of hands on the turn, exactly 44 river cards are possible. So each river child is weighted 1/44. Hands containing that river card are zeroed in both reach vectors before descending, and their values are zeroed on the way back up. Every consistent (hand, hand, river) triple then gets exactly the right probability.

Terminal nodes in linear time

The cost of a vector traversal sits at its leaves. A naive showdown compares each of n hands against each of m opponent hands, which is O(nm), roughly 100,000 operations per terminal. Johanson et al. (2011) show how to do it in O(n + m) once hands are sorted by strength. The sort happens once per river board, using the milestone 3 evaluator.

Walk the traverser's hands from weakest to strongest. Keep a pointer into the opponent's sorted hands, and a running total of the reach of opponent hands strictly weaker than the current one. That total is almost the win weight, except that it includes opponent hands sharing a card with ours, which cannot coexist with it. So the sweep also keeps 52 per-card running totals. The compatible win weight is

$$\text{win}(h) = \text{total} - \text{cardsum}[c_1] - \text{cardsum}[c_2]$$

where c_1 and c_2 are the cards of h. A second sweep from the strongest end gives the loss weight. The value is the stake times (win − loss). Ties contribute nothing, because both players have put in the same amount.

Fold nodes use the same idea: inclusion-exclusion. The opponent reach compatible with hand h is the total, minus the reach of hands containing c_1, minus the reach of hands containing c_2, plus the reach of the *identical* combo, which was subtracted twice.

```
*out = total - card_sum[a] - card_sum[b] + reach_of_same_combo;
```

Exact best response, and why it is easy here

Milestone 1 warned that a best responder must choose *one* action per information set, not per history. Here every information set is a (node, hand) pair, and the counterfactual value vector already sums over the opponent's hidden hands. So the best response is simply the **elementwise maximum** over action-value vectors at the

traverser's nodes. It is exact, and costs about the same as one CFR iteration.

Exploitability is $(BR_0 + BR_1) / 2$. The CLI and tests report it as a percentage of the starting pot, which is how commercial solvers report it. A solve to 0.3% of pot is considered good, and 0.1% is excellent.

Gate 1: the polarized river, in closed form

The first test checks the solver against game theory that can be worked out on paper.

On a board of K♥ 9♦ 5♣ 3♠ 2♥, OOP holds either a set of kings (3 combos) or 7-6 for seven-high (16 combos). IP holds only queens, which beat the bluffs and lose to the value hands. OOP may check or bet B into a pot of P; IP may only check behind.

The equilibrium follows from two indifference conditions:

- IP calls exactly often enough that a bluff breaks even. That makes the call frequency P/(P + B). - OOP bluffs exactly often enough that a call breaks even. That makes the fraction of bets that are bluffs B/(P + 2B).

The test runs this for three bet sizes:

Bet size	IP call frequency	OOP bluff combos	Solver
half pot	2/3	1.0	matches to 0.01
pot	1/2	1.5	matches to 0.01
twice pot	1/3	2.0	matches to 0.01

In every case the kings bet 100% of the time, exploitability is below 0.05% of pot, and OOP's expected value matches the closed form $3/19 \times 150 - 50$.

Gate 2: an independent implementation, and mutation testing

The second test re-expresses the same subgame through the milestone 1 `engine::Game` trait. There, chance deals one *specific* pair of hands with probability proportional to $w_0 \times w_1$. Showdowns call the evaluator directly on seven cards. Nothing is vectorized. This implementation shares none of the clever code: no inclusion-exclusion, no sweep, no chance weighting.

The test then takes the vector solver's strategy at an early, badly exploitable iteration and again near convergence. It translates the strategy into the engine's format and requires both implementations to agree to within 0.002 chips on the expected value, both best-response values and exploitability. It runs this on a river spot (two bet sizes, raises and all-in) and on a turn spot (all 48 river runouts). The ranges are chosen so that hands share cards, the two players hold an identical combo, and some

showdowns are ties. Finally, the scalar CFR+ engine independently converges to the same game value.

Choosing those ranges came from a lesson worth recording. To check that the test could actually catch bugs, I injected deliberate bugs into the vector code one at a time and reran it. This technique is called **mutation testing**.

Against the first version of the test I tried three mutations:

- using the wrong river-card probability; - dropping the identical-combo term in inclusion-exclusion; - counting ties as wins.

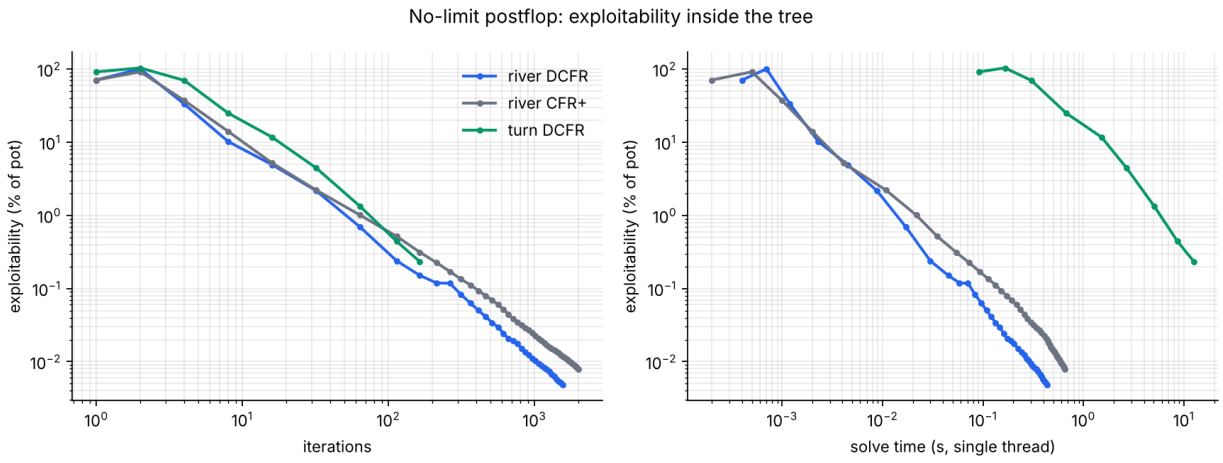
Only the first was caught. The other two passed silently, because the original ranges *never contained an identical combo or a tie*, so those code paths were never exercised. I then added QhJh to both ranges and gave each player a 7-6 suited combo, which tie on this board. The strengthened test catches both of those mutations, and two more: counting ties as losses, and paying the folder's stake instead of the opponent's. A test that cannot fail is not a test.

Gate 3: real spots and invariants

A realistic river spot uses 256 OOP combos against 318 IP combos. The bet menu is 33% and 75% of pot, 60% raises and all-in, with 4× pot behind. It falls from 71% to under 0.3% of pot in 300 iterations. A property test draws random ranges and random bet menus and checks invariants after random iteration counts:

- Every average strategy sums to 1 for every hand at every node. - Nobody invests beyond the stack. - Exploitability is never negative. - The two players' EVs sum to zero. - Each best response is at least as good as the strategy it answers.

Performance



Postflop convergence

Measured single-threaded on a 2-core cloud VM:

Spot	Size	Time to 0.3% of pot	Further
river, 256 × 318 combos	63 nodes, 0.1 MB	0.05 s	0.005% in 0.4 s
turn, 260 × 322 combos	14,799 nodes, 32 MB	13 s	—

On the river, DCFR beats CFR+ at every point on the curve: 0.005% versus 0.008% of pot after about 1,600 versus 2,000 iterations, which matches what we saw on Leduc.

The turn is about 300 times more work than the river, because each of 48 rivers carries its own copy of the river tree. That makes it the target of milestone 5. The 48 river subtrees are independent within an iteration, so they parallelize almost perfectly with rayon; an 8-core M-series laptop should bring the turn solve to a couple of seconds. Suit isomorphism is a further saving: on a two-tone or monotone board, some river cards are strategically identical and need solving only once. Memory will also need attention, because flop solving multiplies the turn tree by about 49.

Try it

```
cargo run --release -p cli -- postflop --board "Qs Jh 7c 4d 2s" \
  --oop "22+,A2s+,K9s+,Q9s+,J9s+,T8s+,97s+,87s,76s,65s,ATo+,KTo+,QTo+,JTo" \
  --ip  "22+,A2s+,K7s+,Q8s+,J8s+,T8s+,97s+,86s+,75s+,64s+,54s,A8o+,KTo+,QTo+,JTo,T9o" \
  --pot 100 --stack 400 --bets 33,75 --raises 60 --target 0.1
```

The output shows exploitability as the solve proceeds, then each player's EV as a share of the pot. It then lists OOP's strategy by hand class, strongest first, and IP's response to each of OOP's first actions.

Reading

Johanson, Waugh, Bowling and Zinkevich (2011), *Accelerating Best Response Calculation in Large Extensive Games*, section 4, for the public-tree, vector-of-ranges structure and the sorted showdown. Brown and Sandholm (2017), *Safe and Nested Subgame Solving*, sections 1 to 3, for why solving a subgame from fixed ranges is "unsafe" inside a larger game, which is the problem milestone 6 has to handle.